

MTP-2 THESIS

Beyond Independent Attribution

Analyzing and Addressing Feature Interaction

Limitations in LIME for Image Classification

Aman Tiwari 24CS60R35

Department of Computer Science and Engineering, IIT Kharagpur

Supervisor: **Prof. Palash Dey**

April 2026

Agenda

What we will cover today

1

Background

What explainability means, why it is needed, and what LIME solves

2

Key Concepts

Synergy, redundancy, and why they matter

3

Proposed Methods

P-LIME, S-LIME, H-LIME — each explained in depth

4

Evaluation

Setup, 5 metrics, and ground truth computation

5

Results & Findings

Aggregate results and computational cost

6

MTP-1 & Wrap-up

Last semester's work, complementary link, future directions

Explainability

What it means, why it matters, and how LIME addresses it

What is explainability? The ability to present a model's prediction in human-understandable terms, identifying which input factors influenced the output.

Here, an explanation = a scored set of image regions (positive → supports class, negative → opposes). Scope: post-hoc, local, model-agnostic.

Why Do We Need It?

- ▶ Neural networks are accurate but opaque — input in, prediction out
- ▶ Without explanations, errors are hard to debug and trust is hard to justify
- ▶ High-stakes domains require auditability and accountability
- ▶ We need to know WHY the model predicted a class, not only WHAT

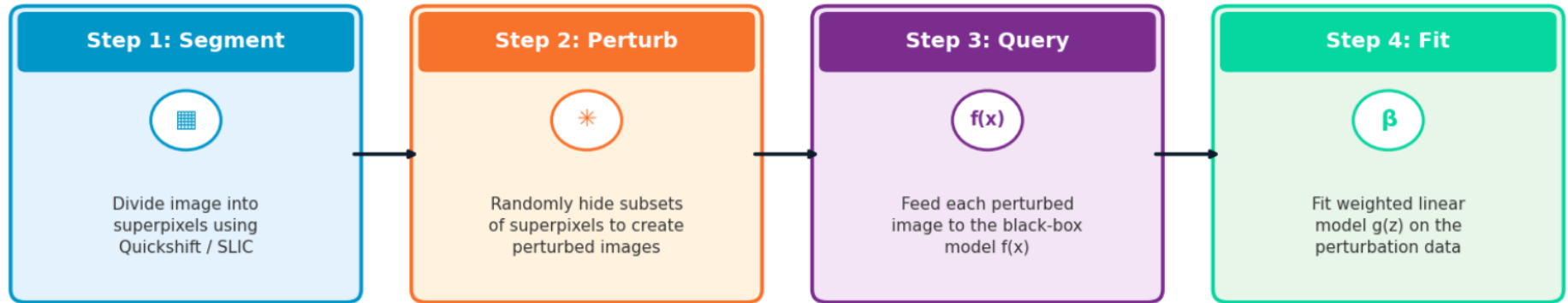
What LIME Provides

- ▶ LIME = Local Interpretable Model-Agnostic Explanations
- ▶ Highlights which image regions caused the prediction
- ▶ Works for ANY black-box model — no internals needed
- ▶ Output: colour map of regions that support or oppose the class

How LIME Works

Local Interpretable Model-Agnostic Explanations (Ribeiro et al., 2016)

LIME Pipeline: Local Interpretable Model-Agnostic Explanations



Output: β_i = importance of superpixel i

Limitation: Step 4 fits $g(z) = \beta_0 + \sum \beta_i z_i$ — no interaction terms $\beta_{ij} z_i z_j \rightarrow$ assumes features are independent

Surrogate model: $g(z) = \beta_0 + \sum \beta_i z_i$

$z_i \in \{0,1\}$ = superpixel present/absent

β_i = importance score for superpixel i

► Linear — no interaction terms $\beta_{ij} z_i z_j$

Example LIME Output



Green = supports predicted class

Red = opposes predicted class

One weight per superpixel — no information about how regions interact.

Two Limitations of LIME

LIME is a great start — but it has two structural problems



Problem 1: Unnatural Perturbations (MTP-1)

- ▶ LIME hides superpixels using black or gray patches
- ▶ These look nothing like natural images — the model has never seen them during training
- ▶ Confidence drops may reflect ‘this looks weird’, not ‘this region is important’
- ▶ Addressed in MTP-1 using VAE-based inpainting (Surface Integrated Gradients)

Problem 2: Independence Assumption (MTP-2 — This Work)

- ▶ LIME assigns one independent score per superpixel
- ▶ Reality: two regions may only matter together (synergy) or overlap in effect (redundancy)
- ▶ LIME’s linear model has no way to capture this joint effect
- ▶ Addressed in MTP-2 using P-LIME, S-LIME, and H-LIME

Synergy & Redundancy

Two types of feature interaction that LIME's linear model cannot capture

Interaction formula: $I(A,B) = \text{Drop}(A \cup B) - \text{Drop}(A) - \text{Drop}(B)$

$I(A,B) > 0$ means synergy | $I(A,B) < 0$ means redundancy

Synergy ($I(A,B) > 0$)

- ▶ Two features are weak alone, but strong together
- ▶ Combined drop is more than the individual drops added together
- ▶ LIME underestimates because it treats them independently
- ▶ Example: ear + face jointly trigger the dog prediction

Positive example: $\text{Drop}(A \cup B) > \text{Drop}(A) + \text{Drop}(B)$

Redundancy ($I(A,B) < 0$)

- ▶ Two features carry overlapping information
- ▶ Combined drop is less than the individual drops added together
- ▶ LIME overestimates because it counts similar evidence twice
- ▶ Example: left eye + right eye both support the cat prediction

Negative example: $\text{Drop}(A \cup B) < \text{Drop}(A) + \text{Drop}(B)$

Synergy Problem in Action

Golden Retriever — hiding ear and face individually vs together



Hide Region A (ear)

Confidence drops by only 0.011

Barely affects the prediction

Hide Region B (face)

Confidence drops by 0.132

Moderate effect on its own

Hide BOTH A+B

Confidence drops by 0.686!

LIME expects: 0.143 | Actual: 0.686

Interaction = +0.543 (massive synergy)

Redundancy Problem in Action

Golden Retriever — real negative interaction found by pairwise search

LIME Interaction Problem - golden retriever
Redundancy $I(A,B) = -0.0814$ -> LIME overestimates both regions!

Original
Conf = 0.9253



Hide Region A
Conf = 0.9178 (drop = 0.0076)



Hide Region B
Conf = 0.6505 (drop = 0.2748)



Hide Both A+B
Conf = 0.7244 (drop = 0.2010)
LIME expects: 0.2824 Actual: 0.2010



Hide Region A

Confidence drops by 0.0076
Small positive evidence region

Hide Region B

Confidence drops by 0.2748
Strong positive evidence region

Hide BOTH A+B

Actual joint drop is 0.2010
LIME expects: 0.2824 | Actual: 0.2010
Interaction = -0.0814 (real redundancy)

Method 1: P-LIME (Pairwise LIME)

Add quadratic interaction terms to the surrogate model | $\sim 8\times$ LIME cost

Core Idea

- LIME: $g(z) = \sum \beta_i \cdot z_i$ (independent features only)
- P-LIME: $g(z) = \beta_0 + \sum \beta_i z_i + \sum \beta_{ij} z_i z_j$ (+ pairwise terms)
- 561 pair columns for 34 superpixels, Ridge regression
- 10,000 perturbation samples (vs 1,000 for LIME)

Equation Terms

- z_i : 1 if superpixel i is visible, 0 if hidden
- β_i : individual effect of superpixel i
- $z_i z_j$: active only when both i and j are visible
- β_{ij} : interaction effect between superpixels i and j

How to read β_{ij} : **it tells whether two superpixels interact**

$$\beta_{ij} > 0$$

synergy: pair is stronger together

$$\beta_{ij} < 0$$

redundancy: pair overlaps in information

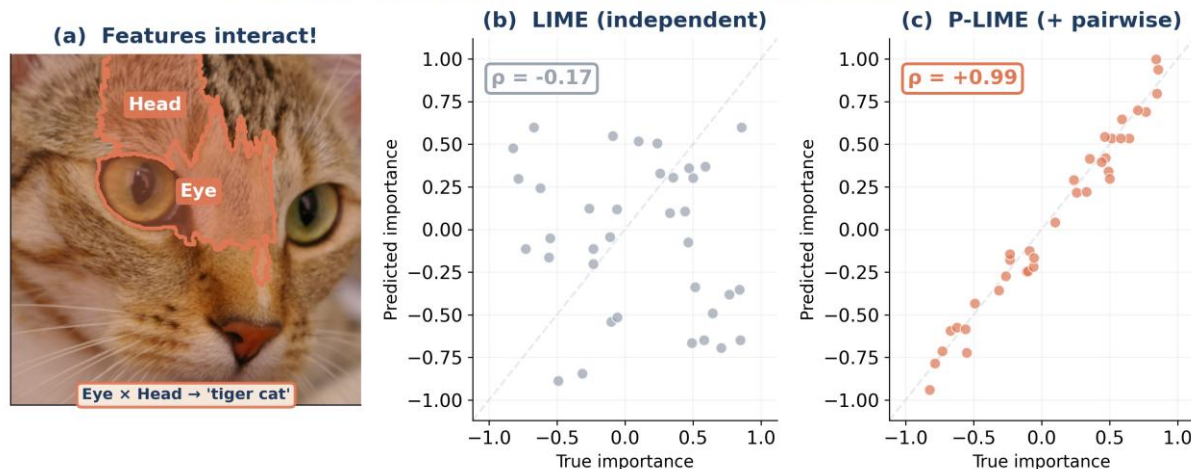
$$\beta_{ij} \approx 0$$

independent: standard LIME was enough

P-LIME: Reading the Image and Plot

The visual shows that there is interaction between the eye and head regions

P-LIME captures feature interactions that LIME misses



Left: image regions

The eye and head regions interact. Together they strongly support the tiger-cat prediction.

Middle: LIME plot

Standard LIME treats regions independently, so predicted importance does not match true importance well.

Right: P-LIME plot

After adding pairwise terms, predicted and true importance align much better. This shows interaction is captured.

Main takeaway: P-LIME keeps LIME interpretable, but adds pairwise terms so it can capture interaction.

Method 2: S-LIME (Shapley LIME)

Replace regression with Shapley value estimation | $\sim 19\times$ LIME cost

Core Idea

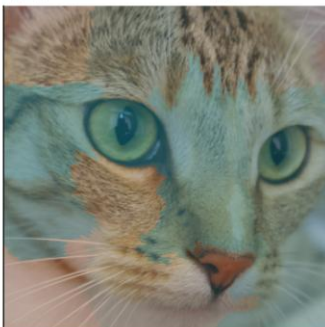
- Replace LIME's regression with Shapley values
- Average marginal contribution across many orderings
- 1,000 random permutations (Monte Carlo sampling)
- Considers all coalitions, not just pairs

Why It Handles Interactions

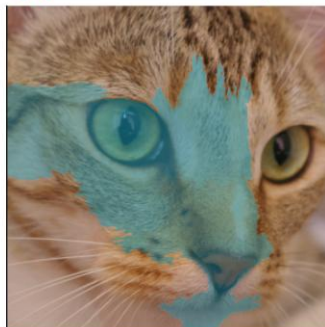
- Each superpixel is evaluated in many different contexts
- The average naturally shares credit across interacting regions
- No explicit pairwise equation is required
- More expensive, but gives fairer feature credit

S-LIME: Shapley values give each superpixel a fair share of credit

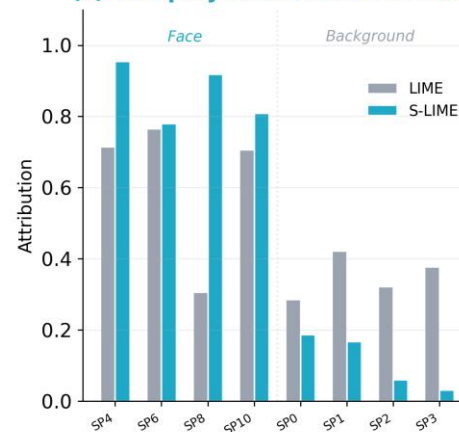
(a) LIME — noisy, unfair



(b) S-LIME — clean, fair



(c) Shapley re-distributes credit



LIME noisy heatmap vs S-LIME clean heatmap — Shapley re-distributes credit fairly to the face

S-LIME: What the Visual Shows

Use this slide to explain the image comparison directly

S-LIME: Shapley values give each superpixel a fair share of credit

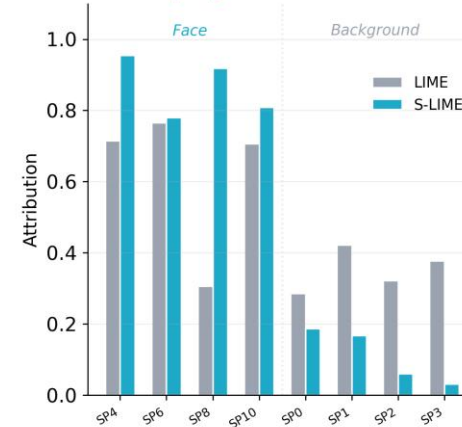
(a) LIME — noisy, unfair



(b) S-LIME — clean, fair



(c) Shapley re-distributes credit



Left: Standard LIME

Noisy attribution map; important evidence is scattered because LIME fits one local linear model.

Middle: S-LIME

Cleaner attribution; credit is averaged over many feature orderings, so the main object regions become clearer.

Right: Comparison chart

Quantitative check: S-LIME improves top-region ranking because it distributes credit more fairly.

Main takeaway: S-LIME keeps the explanation focused on regions that consistently contribute across many contexts.

Method 3: H-LIME (Hierarchical LIME)

Fuse LIME at multiple segmentation scales | $\sim 3.5\times$ LIME cost

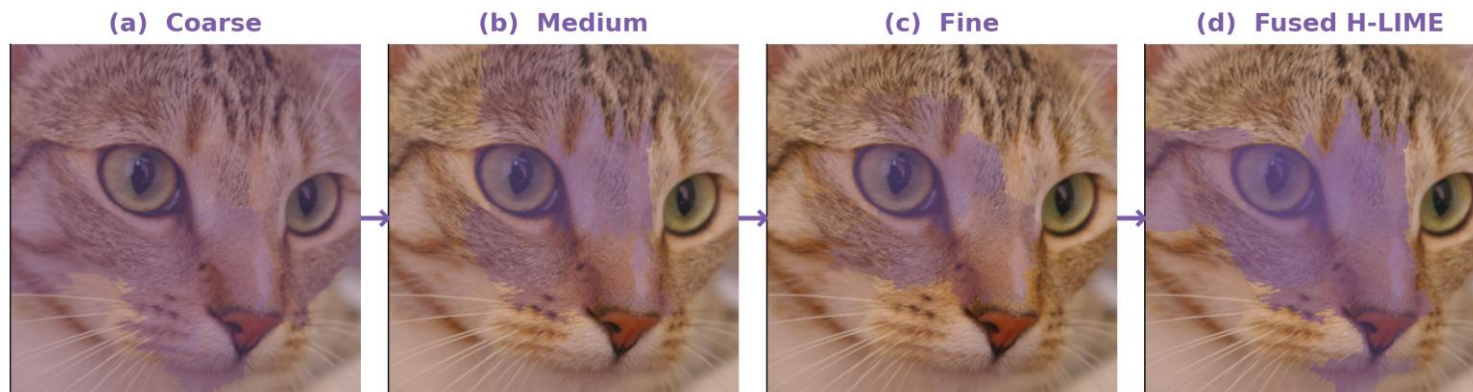
Core Idea

- ▶ Run LIME at 3 scales: coarse (10–15), medium (25–35), fine (50–60)
- ▶ Combine: $w_1 \cdot \text{coarse} + w_2 \cdot \text{medium} + w_3 \cdot \text{fine}$
- ▶ Optimize w_1, w_2, w_3 via deletion fidelity
- ▶ $3\times$ LIME runs total, weights found by grid search

Why It Handles Interactions

- ▶ Coarse scale: interacting SPs merge into one region
- ▶ Fine scale: captures pixel-level detail
- ▶ Fusion recovers both broad dependencies and fine detail
- ▶ Best Deletion AUC: tiger cat (0.099) and aggregate (0.114)

H-LIME: multi-scale explanations fused into a sharp, faithful map



Coarse $\rightarrow$ Medium $\rightarrow$ Fine $\rightarrow$ Fused attribution map — sharp where it matters, smooth elsewhere

H-LIME: Reading the Multi-Scale Image

Explain the visual from left to right

H-LIME: multi-scale explanations fused into a sharp, faithful map



Same H-LIME visual: coarse / medium / fine explanations combine into the final fused map

Coarse scale

Large regions such as head, body, and background. Captures broad object-level evidence.

Medium scale

Breaks the object into meaningful parts such as ear, eye, face, and fur patches.

Fine scale

Adds local detail and sharp boundaries, but can become noisy if used alone.

Fused map

Combines all scales so broad evidence and fine detail both survive in one final explanation.

Main takeaway: H-LIME works because interacting superpixels often appear together at coarser scales, then fine scales restore precision.

Experimental Setup

Aggregate evaluation and ground-truth construction

Aggregate evaluation protocol

- Dataset: ImageNet-V2 — same 1,000 ImageNet classes, independently collected and harder than ImageNet validation
- Model: ResNet-50 pretrained on ImageNet (standard benchmark classifier)
- Segmentation: Quickshift algorithm (kernel=4, max_dist=200, ratio=0.2) → around 34 superpixels per image
- Methods compared: LIME baseline, P-LIME, S-LIME, and H-LIME
- Report: mean scores over 100 randomly sampled ImageNet-V2 images — not separate case-study results

Ground Truth: what is the correct importance of a superpixel?

1. Individual drop

Hide superpixel i alone

$$\Delta_i = f(\mathbf{x}) - f(\mathbf{x} \text{ without } i)$$

2. Pair interaction

Hide i and j together

$$I(i,j) = \Delta_{ij} - \Delta_i - \Delta_j$$

3. True importance

Main effect + shared interactions

$$\text{true}_i = \Delta_i + \frac{1}{2} \sum_j I(i,j)$$

This gives a gold standard that includes both direct superpixel effects and pairwise interaction effects.

Evaluation Metrics — Part 1

Three basic checks: remove, restore, and sign regions correctly

1

Deletion AUC

Lower is better ↓

Question: If we remove the regions marked important, does model confidence fall quickly?

Procedure: rank pixels by importance → remove most important first → plot confidence after each removal.

Good explanation: confidence drops early, so the area under the deletion curve is small.

2

Insertion AUC

Higher is better ↑

Question: If we start from a blank image and add important regions back, does confidence recover quickly?

Procedure: start with a mean-colour image → insert pixels from most to least important → plot confidence rise.

Good explanation: confidence rises early, so the area under the insertion curve is large.

3

Sign Agreement

Higher is better ↑

Question: Does the method agree with ground truth on whether a region supports or opposes the class?

Procedure: compare $\text{sign}(\text{method score})$ with $\text{sign}(\text{true importance})$ for every superpixel.

Formula: $SA = (1/N) \sum 1[\text{sign}(\text{score}_i) = \text{sign}(\text{true}_i)]$.

Evaluation Metrics — Part 2

Ranking quality and interaction-awareness

4

Top-K Jaccard (↑ higher is better)

Pick the top-5 most important superpixels according to the METHOD. Pick the top-5 according to the GROUND TRUTH. Jaccard similarity = how much these two sets overlap.

Formula: $J = |\text{TopK_method} \cap \text{TopK_true}| / |\text{TopK_method} \cup \text{TopK_true}|$

Example: Ground truth top-5 = {Eye, Ear, Nose, Whiskers, Forehead}. Method says = {Eye, Ear, Tail, Background, Paw}. Overlap = {Eye, Ear} = 2. Union = 8 elements. Jaccard = $2/8 = 0.25$ (poor). Perfect = 1.0.

5

Joint Deletion / Interaction Fidelity (↑ higher is better)

Take the top-5 superpixels according to the method and hide them ALL at the same time. Measure how much the model confidence drops.

Formula: $IF = (f(\text{original}) - f(\text{hide top-5 together})) / f(\text{original})$

This is different from Deletion AUC! Deletion AUC removes pixels one-by-one at pixel level. Joint Deletion removes 5 superpixels all at once — this tests whether the method found regions whose JOINT removal is most damaging.

Aggregate Results — 100 ImageNet-V2 Images

Mean scores over 100 randomly sampled images | ResNet-50

Method	Del. AUC ↓	Ins. AUC ↑	Sign ↑	Top-K ↑	Jnt. Del. ↑
LIME	0.120	0.632	62.7%	0.367	0.736
P-LIME	0.118	0.671	66.0%	0.364	0.761
S-LIME	0.117	0.663	64.4%	0.381	0.781
H-LIME	0.114	0.629	63.4%	0.320	0.802

- ▶ **P-LIME:** Best Insertion AUC (0.671, +6.2%) and Sign Agreement (66.0%) — pairwise interactions improve faithfulness at scale
- ▶ **H-LIME:** Best Deletion AUC (0.114) and Joint Deletion (0.802) — multi-scale hierarchy captures what others miss
- ▶ **S-LIME:** Best Top-K Jaccard (0.381) — Shapley sampling reliably identifies top features
- ▶ All 3 proposed methods beat standard LIME on most metrics — interaction-aware surrogates generalize

Computational Cost

Wall-clock time per image | Tesla T4 GPU

Method	Time	Relative	Key Cost Driver
LIME	~10s	1×	1,000 perturbation samples
P-LIME	~78s	~8×	10,000 samples for $O(N^2)$ interaction parameters
S-LIME	~189s	~19×	1,000 permutations × N forward passes each
H-LIME	~35s	~3.5×	3 LIME runs (one per scale) + weight optimization

- ▶ **P-LIME:** Moderate cost (8×) with best faithfulness — wins Insertion AUC and Sign Agreement
- ▶ **S-LIME:** Most expensive (19×) — best Top-K Jaccard but diminishing returns on other metrics
- ▶ **H-LIME:** Best cost-performance ratio at 3.5× — wins Deletion AUC and Joint Deletion

Generative AI was used for code debugging and refinement. Problem formulation, method design, experiments, and analysis were done independently.

Key Findings



All proposed methods outperform standard LIME

LIME wins no metric exclusively; the independence assumption is clearly suboptimal



P-LIME is the strongest overall performer

Wins 2 metrics: best Insertion AUC (0.671, +6.2%) and Sign Agreement (66.0%) — pairwise interactions matter most



H-LIME is the best at identifying critical regions

Wins 2 metrics: best Deletion AUC (0.114) and Joint Deletion (0.802) — multi-scale captures what others miss



S-LIME best identifies top features

Best Top-K Jaccard (0.381) — Shapley values reliably rank the most important superpixels



Findings generalize to a 100-image aggregate evaluation

On ImageNet-V2: all 3 proposed methods beat LIME on Insertion AUC, Sign Agreement, and Joint Deletion

MTP-1: The Perturbation Problem

Last semester's work — a different limitation of LIME

How LIME Perturbs Images

- ▶ LIME hides superpixels using black or gray patches
- ▶ These artificial images look nothing like natural images
- ▶ ResNet-50 has never seen such inputs during training
- ▶ Model is operating out-of-distribution (OOD)
- ▶ Even hiding irrelevant pixels causes unexpected confidence drops
- ▶ Attribution scores become unreliable because of distribution shift

Why This Is a Problem

- ▶ Confidence drops reflect distribution shift, not true importance
- ▶ LIME confuses “this pixel matters” with “this looks weird”
- ▶ Affects LIME Step 2: Perturbation generation

Example

Hiding a grass patch with black creates an unnatural boundary. The model's confidence drops not because grass matters, but because it has never seen a black rectangle in a photo.

Result: False attribution to irrelevant regions.

What is a Variational Autoencoder (VAE)?

A generative model that learns the space of natural images

Basic Idea

- ▶ Encoder compresses an image into a small latent vector z
- ▶ Decoder reconstructs an image back from z
- ▶ During training, the latent space is made smooth and continuous
- ▶ Nearby points in latent space decode to visually similar, realistic images

Image → Encoder → z → Decoder → Realistic Image

Why VAE Helps MTP-1

- ▶ Standard LIME creates black-patch images that look unnatural
- ▶ A VAE can generate realistic alternatives instead of black patches
- ▶ Perturbations stay on the natural image manifold
- ▶ So classifier confidence changes are more meaningful
- ▶ This fixes the perturbation-generation problem from MTP-1

One-line takeaway: VAE replaces artificial black patches with realistic image variations.

MTP-1: Solution — Surface Integrated Gradients

VAE-based attribution that stays on the natural image manifold

Key Idea: Stay on the Manifold

- ▶ Train a Variational Autoencoder (VAE) on natural images
- ▶ VAE learns a smooth latent space where every point → realistic image
- ▶ Instead of black-patch occlusion, perturb in VAE latent space
- ▶ All perturbed images stay in-distribution for the classifier
- ▶ Confidence estimates now reflect true feature importance

Surface Integrated Gradients (SIG)

- ▶ Integrate attributions over a 2D surface in latent space
- ▶ Uses top principal components of VAE posterior
- ▶ More robust than single-path integration (MIG)

Standard LIME vs MTP-1

LIME Perturbation: Replace superpixel with black patch

→ *Artificial, out-of-distribution image*

→ *Model has never seen this during training*

MTP-1 (SIG): Perturb in VAE latent space

→ *Realistic, on-manifold image*

→ *Model operates in familiar input space*

LIME Pipeline Stage Affected: Step 2 (Perturbation Generation)

MTP-1 + MTP-2: Two Complementary Fixes for LIME

Together they address both sources of error

MTP-1

Fix Perturbation Quality

- Problem: Black-patch perturbations are out-of-distribution
- Solution: VAE-based realistic inpainting (SIG)
- Fixes: Step 2 — Perturbation generation
- Result: More reliable confidence estimates



MTP-2

Fix Independence Assumption

- Problem: Linear model ignores feature interactions
- Solution: P-LIME, S-LIME, H-LIME
- Fixes: Step 4 — Surrogate model fitting
- Result: More faithful attribution maps

Key Insight: Completely Orthogonal Improvements

- MTP-1 improves the input quality (better perturbations → better data for the surrogate)
- MTP-2 improves the model structure (interaction-aware surrogate → better fit on the data)
- Combining both would address both sources of error simultaneously → natural future direction

Future Work

Extension to other domains Apply interaction-aware attribution to text, tabular, and time-series LIME

Larger-scale evaluation Test on more images, more models (ViT, EfficientNet), more datasets

Thank You

Questions & Discussion

Aman Tiwari

24CS60R35 | Department of CSE, IIT Kharagpur

Supervisor: **Prof. Palash Dey**